

COSMOS-Q: Cognitive Memory for Qwen Agents

1 Goal

Persistent memory is essential for longitudinal LLM agents. However, many existing memory systems treat memory as an append-only database with similarity-based retrieval. This often leads to stale memory retrieval, unbounded memory growth, and poor adaptation to changing user preferences or task contexts.

We propose **COSMOS-Q**, a cognitive memory architecture for Qwen-powered agents. COSMOS-Q treats memory as an adaptive lifecycle rather than a static store. The system is built around four mechanisms:

1. **Versioned Retrieval-Triggered Reconsolidation (RTR)**: Retrieved memories become re-visible when new context creates sufficient prediction error. Prior versions are preserved and superseded rather than overwritten.
2. **Asynchronous Schema Consolidation (ASC)**: Episodic memories are periodically abstracted into higher-level schemas representing stable preferences, goals, facts, behaviors, and procedures.
3. **Interference-Aware Adaptive Forgetting (IAAF)**: Memories are demoted, archived, or compressed based on semantic interference, contradiction, redundancy, and instability rather than simple recency decay.
4. **Utility-Aware Context Packing (UACP)**: Candidate memories are selected under a token budget using utility signals derived from the memory lifecycle, including relevance, stability, schema membership, interference, recency, and token cost.

The central hypothesis is:

Longitudinal LLM agents perform better when memory is managed as a cognitive lifecycle involving reconsolidation, consolidation, interference-aware forgetting, and utility-aware retrieval, rather than as flat similarity-based storage.

Given a user query q , agent state s , memory set $\mathcal{M}$, and context budget B , COSMOS-Q selects a subset of memories $\mathcal{S} \subseteq \mathcal{M}$:

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{M}} \sum_{m_i \in \mathcal{S}} U(m_i | q, s) \quad \text{subject to} \quad \sum_{m_i \in \mathcal{S}} T(m_i) \leq B, \quad (1)$$

where $T(m_i)$ denotes the token cost of memory m_i .

The memory utility function is defined as:

$$U(m_i | q, s) = R(m_i, q) \cdot S(m_i) \cdot (1 - I(m_i)) + B_{\text{schema}}(m_i) + B_{\text{recency}}(m_i) - \lambda T(m_i), \quad (2)$$

where R is relevance, S is stability, I is interference, B_{schema} is schema membership bonus, B_{recency} is recency bonus, and λ controls token-cost penalty.

2 Architecture

COSMOS-Q contains six primary components: an episodic buffer, a schema store, a retrieval and reconsolidation module, a utility-aware context packing module, an asynchronous consolidation engine, and an interference-aware forgetting engine.

2.1 System Overview

```
User Query
  |
  v
Qwen Agent Runtime
  |
  v
COSMOS-Q Memory Layer
  |
  |-- Episodic Buffer
  |-- Schema Store
  |-- Retrieval + Versioned Reconsolidation
  |-- Utility-Aware Context Packing
  |-- Asynchronous Schema Consolidation
  |-- Interference-Aware Forgetting
  |
  v
Memory Brief
  |
  v
Qwen Response
  |
  v
Trace Logger -> Memory Update Pipeline
```

2.2 Memory Representation

Each memory is represented as a versioned memory node.

```
MemoryNode {
  id: UUID
  user_id: UUID
  version: int
  content: string
  embedding: vector
  evidence: List[ContextRef]
  stability: float
  interference_score: float
  schema_id: UUID or null
  parent: UUID or null
  successor: UUID or null
  status: ACTIVE, SUPERSEDED, CONSOLIDATED, or ARCHIVED
  created_at: datetime
  updated_at: datetime
  reconsolidation_count: int
}
```

Schemas are higher-level abstractions derived from episodic memories.

```
Schema {
  id: UUID
  type: PREFERENCE, GOAL, FACT, PROCEDURE, or BEHAVIOR
  content: string
  confidence: float
  supporting_memories: List[UUID]
  contradicting_memories: List[UUID]
  version: int
  created_at: datetime
  updated_at: datetime
}
```

2.3 Versioned Retrieval-Triggered Reconsolidation

When a memory m_v is retrieved, COSMOS-Q compares the memory’s expected implication with the current context c . If the prediction error exceeds a threshold τ_{rtr} , a new memory version is created.

$$\Delta = D_{\text{sem}}(P(m_v, q), E(c)), \quad (3)$$

where $P(m_v, q)$ is the expected implication of memory m_v for query q , $E(c)$ is information extracted from the current context, and D_{sem} is semantic divergence.

If $\Delta < \tau_{\text{rtr}}$, the memory is reinforced:

$$S(m_v) \leftarrow \min(1, S(m_v) + \alpha_{\text{reinforce}}). \quad (4)$$

If $\Delta \geq \tau_{\text{rtr}}$, COSMOS-Q creates a new memory version:

$$m_{v+1} = \text{Merge}(m_v, c, \Delta). \quad (5)$$

The previous version is retained but marked as superseded:

$$\text{Status}(m_v) = \text{SUPERSEDED}, \quad \text{Successor}(m_v) = m_{v+1}. \quad (6)$$

This preserves provenance and enables temporal reasoning over memory evolution.

2.4 Asynchronous Schema Consolidation

Between sessions, COSMOS-Q consolidates episodic memories into schemas. The replay batch combines recent memories, stable memories, and unstable memories:

$$\mathcal{R} = \mathcal{M}_{\text{recent}} \cup \mathcal{M}_{\text{stable}} \cup \mathcal{M}_{\text{unstable}}. \quad (7)$$

For each semantic cluster $C \subseteq \mathcal{R}$, the system finds the closest existing schema:

$$g^* = \arg \max_{g \in \mathcal{G}} \text{Sim}(C, g), \quad (8)$$

where $\mathcal{G}$ is the schema store.

If C is consistent with g^* , the schema is refined:

$$g^* \leftarrow \text{RefineSchema}(g^*, C). \quad (9)$$

If C contradicts g^* , schema confidence is reduced:

$$\text{Conf}(g^*) \leftarrow \text{Conf}(g^*) - \alpha_{\text{contradict}}. \quad (10)$$

If confidence falls below a split threshold, the schema is split:

$$\text{if } \text{Conf}(g^*) < \tau_{\text{split}}, \quad g^* \rightarrow \{g_1, g_2, \dots, g_k\}. \quad (11)$$

2.5 Interference-Aware Adaptive Forgetting

COSMOS-Q computes interference between semantically similar memories. For memory m_i , interference is estimated over nearest neighbors $\mathcal{N}(m_i)$:

$$I(m_i) = \sum_{m_j \in \mathcal{N}(m_i)} \text{Sim}(m_i, m_j) \cdot \text{Contradict}(m_i, m_j) \cdot \frac{1}{1 + |t_i - t_j|}. \quad (12)$$

The forgetting score is:

$$F(m_i) = \frac{I(m_i)}{1 + S(m_i)}. \quad (13)$$

If $F(m_i) > \tau_{\text{forget}}$, the memory is archived or compressed into its parent schema before removal from active retrieval:

$$\text{Status}(m_i) \leftarrow \text{ARCHIVED}. \quad (14)$$

2.6 Utility-Aware Context Packing

Given candidate memories $\mathcal{C}$, COSMOS-Q selects a memory subset under a token budget. Each memory is ranked by utility per token:

$$\rho(m_i) = \frac{U(m_i \mid q, s)}{T(m_i)}. \quad (15)$$

Candidates are sorted by $\rho(m_i)$ in descending order and greedily packed until the token budget B is exhausted. The selected memories are then composed into a compact memory brief and passed to the Qwen agent.

3 Experiment Plan

3.1 Research Questions

We evaluate five research questions:

1. **RQ1:** Does COSMOS-Q improve cross-session task performance compared to standard memory baselines?
2. **RQ2:** Does versioned reconsolidation improve adaptation to changing or contradictory user preferences?
3. **RQ3:** Does schema consolidation reduce memory footprint while preserving useful long-term knowledge?

4. **RQ4:** Does interference-aware forgetting reduce stale or contradictory memory retrieval?
5. **RQ5:** Does utility-aware context packing improve token efficiency compared to top- k similarity retrieval?

3.2 Benchmarks

We evaluate COSMOS-Q on public and custom longitudinal memory benchmarks.

Benchmark	Focus	Capability Tested
LoCoMo	Multi-session conversations	Temporal reasoning, personalization
LongMemEval	Long-range memory QA	Knowledge updates, abstention
C-MEM	Controlled memory scenarios	Contradictions, forgetting, schema drift

Table 1: Evaluation benchmarks.

The custom benchmark, C-MEM, contains controlled multi-session user trajectories with stable preferences, changing preferences, temporary instructions, project-specific facts, contradictory updates, stale information, and procedural learning cases.

3.3 Baselines

COSMOS-Q is compared against the following baselines:

1. **No Memory:** The agent only sees the current query.
2. **Full Transcript:** The agent receives all available history until the context window is filled.
3. **Rolling Summary:** Prior interactions are compressed into a running summary.
4. **Naive RAG:** Top- k memories are retrieved using cosine similarity.
5. **Recency Retrieval:** The most recent memories are retrieved.
6. **COSMOS-Q without RTR.**
7. **COSMOS-Q without ASC.**
8. **COSMOS-Q without IAAF.**
9. **COSMOS-Q without UACP.**
10. **Full COSMOS-Q.**

3.4 Metrics

We evaluate performance across four categories.

Category	Metrics
Task Performance	Accuracy, F1, LLM-as-judge score, task success rate
Personalization	Preference adherence, contradiction resolution accuracy
Memory Quality	Recall@k, Precision@k, stale memory retrieval rate
Efficiency	Tokens retrieved, memory footprint, latency, compression ratio

Table 2: Evaluation metrics.

3.5 Ablation Study

To isolate the contribution of each mechanism, we evaluate:

$$\text{COSMOS-Q}_{\text{full}} \text{ vs. } \text{COSMOS-Q}_{\text{-RTR}}, \text{ COSMOS-Q}_{\text{-ASC}}, \text{ COSMOS-Q}_{\text{-IAAF}}, \text{ COSMOS-Q}_{\text{-UACP}}. \quad (16)$$

Expected effects:

- Removing RTR should reduce adaptation to changed preferences.
- Removing ASC should increase memory footprint and reduce abstraction quality.
- Removing IAAF should increase stale and contradictory memory retrieval.
- Removing UACP should reduce token efficiency and retrieval precision.

3.6 Expected Outcomes

We expect COSMOS-Q to show:

1. Higher cross-session task success than summary-based and vector-only memory.
2. Lower stale memory retrieval rate due to interference-aware forgetting.
3. Better adaptation to preference changes due to versioned reconsolidation.
4. Lower active memory growth due to schema consolidation.
5. Higher utility per retrieved token due to utility-aware context packing.